UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER II

2025/2026

SECP 3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 02

Tutorial 3 - AI Algorithm

LECTURER: DR. ARYATI BINTI BAKRI

NAME	MATRIC NO
NUR FIRZANA BINTI BADRUS HISHAM	A23CS0156
NURAISYAH BINTI MOHD ZIKRE	A23CS0160
WU CHEN XI	X25EC3007

What is the meaning of Image Classification?

Image classification is a computer vision problem, defining images as belonging to a particular class. The model is fed an image and predicts the class of the image. For instance, it can detect if a picture shows a cat, dog, airplane or car. We have used the CIFAR-10 dataset with 10 image categories in this project. The process of image classification has numerous applications including medical imaging, self-driving cars, and facial recognition.

What is CNN?

The Convolutional Neural Network (CNN) is a deep learning model used for image processing. CNN is able to extract all important characteristics of an image, including shapes, edges and textures. It features convolution layers to learn features from the image and pooling layers to squeeze out image dimensions while retaining useful content. In this project, we use a CNN model to classify images from the CIFAR-10 dataset. CNN out performs a traditional neural network because it can more effectively extract image features.

Evaluation of CNN

The CNN model is tested with the test data. The primary measure of evaluation is accuracy, defined as the number of images the model classifies correctly. Also, a confusion matrix is used to compare the predicted labels with the actual labels. The CNN model was able to reach approximately 70% accuracy, outperforming the ANN model with 49% accuracy. The result indicates CNN's superior performance for image classification as it can learn image features more efficiently.

Steps hand on in Python

This section documents the step-by-step hands-on implementation of an image classification system using CNN on the CIFAR-10 dataset in Python. The complete code was executed in a Google Colab environment using TensorFlow and Keras.

STEP 1: Import the required libraries

Figure 1 shows the libraries that are imported to support the model building which is Convolutional Neural Network (CNN), data handling, and evaluation.

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
```

```
import seaborn as sns
```

Figure 1: Python Code for Imported Libraries

TensorFlow or Keras provides a deep learning framework. NumPy handles numerical operations, Matplotlib and Seaborn handle visualization. Sckit-learn provides the evaluation of the metrics.

STEP 2: Load and Explore the CIFAR-10 datasets

CIFAR-10 is a benchmark dataset that contains 60,000 colour images of size 32x32 pixels across 10 classes. The classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. There are 50,000 training images and 10,000 test images to be tested by loading this data into python as in Figure 2.

```
(X_train, y_train), (X_test, Y_test) = datasets.cifar10.load_data()
```

```
y_train = y_train.reshape(-1,)
y_train[:5]
```

```
array([6, 9, 9, 4, 1], dtype=uint8)
```

```
y_test = Y_test.reshape(-1,)
```

```
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog',
           'frog', 'horse', 'ship', 'truck']
```

Figure 2: Load the CIFAR-10 Dataset

The training set has shape (50000, 32, 32, 3) and the test set has shape (10000, 32, 32, 3) like shown in Figure 3. Labels are reshaped from 2D to 1D arrays for compatibility with sparse categorical cross-entropy loss.

```
X_train.shape
```

```
(50000, 32, 32, 3)
```

```
X_test.shape
```

```
(10000, 32, 32, 3)
```

Figure 3: Train and Test Set Result

STEP 3: Visualize the Sample Images

A helper function is defined to visualize individual images and their corresponding class labels. The code in Figure 4 shows how to visualize the sample image.

```
def plot_sample(X, y, index):
  plt.figure(figsize=(15, 2))
  plt.imshow(X[index])
  plt.xlabel(classes[y[index]])
```

Figure 4: Visualize the Sample Image

Figure 5 shows the sample image result that plots the image of automobile with set train 5 and ship image with set train 501.

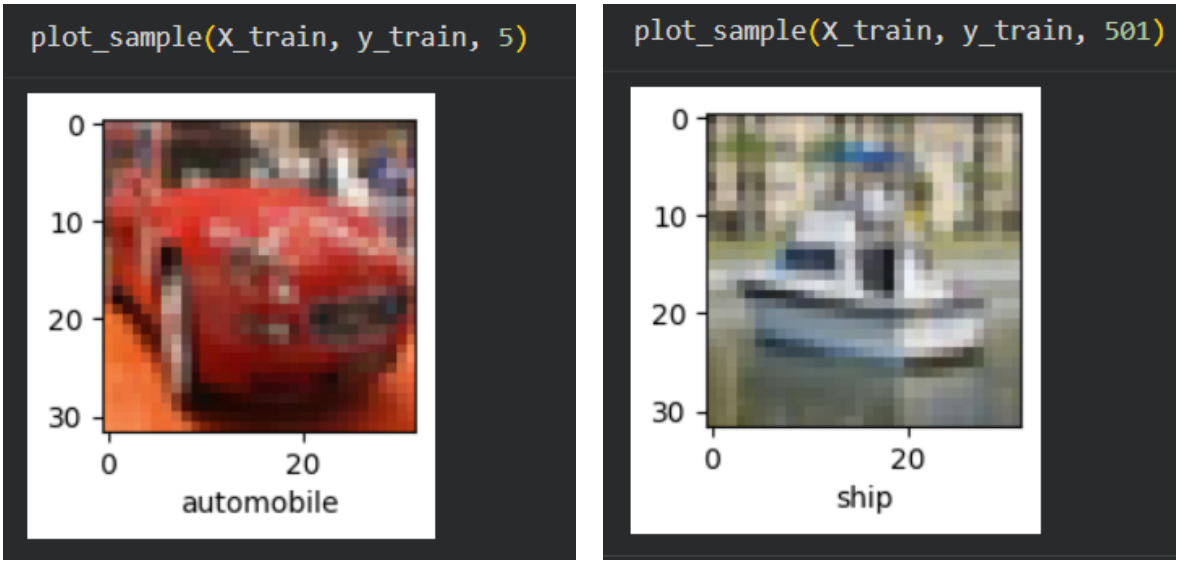

Figure 5: The Output of the Sample Image

STEP 4: Normalize the Pixel Values

Pixel values are originally in the range [0, 255], so they are scaled to [0.0, 1.0] as in Figure 6 to speed up the training and improve the gradient stability.

```
X_train = X_train /255.0
X_test = X_test / 255.0
```

Figure 6: Code to Normalize Pixel Values

STEP 5: Build the ANN Baseline

Before building the CNN, a simple Artificial Neural Network (ANN) is trained as a baseline like in Figure 7 to highlight the advantage of convolutional layers for image data.

```
ann = models.Sequential([
    layers.Flatten(input_shape = (32,32,3)),
    layers.Dense(3000, activation = 'relu'),
    layers.Dense(1000, activation = 'relu'),
    layers.Dense(10, activation = 'softmax')
])

ann.compile(optimizer = 'SGD',
            loss= 'sparse_categorical_crossentropy',
            metrics = ['accuracy'])

ann.fit(X_train, y_train, epochs = 5)
```

Figure 7: ANN Baseline Code

The ANN uses SGF optimizer over 5 epochs. The Flatten layer converts 32x32x3 images into a 1D vector of 3,072 values. Despite having 3,000 and 1,000 dense neurons, the ANN typically achieves only about 50-55% test accuracy because it ignores the spatial structure of images.

STEP 6: Building the Original CNN Model

The CNN model introduces convolutional and pooling layers to learn the spatial features.

```
cnn = models.Sequential([
    layers.Conv2D(filters = 32, kernel_size = (3,3),
                  activation = 'relu', input_shape = (32,32,3)),
    layers.MaxPooling2D((2,2)),

    layers.Conv2D(filters = 64, kernel_size = (3,3),
                  activation = 'relu'),
    layers.MaxPooling2D((2,2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

cnn.fit(X_train, y_train, epochs=10)
```

Figure 8: Original CNN Model Build

The first Conv2D layer applies 32 filters (3x3 kernels) to detect low-level features such as edges and corners. MaxPooling2D reduces spatial dimensions by half, remaining the most dominant features. The second Conv2D with 64 filters learns higher-level patterns. The Dense(64) layer performs classification based on these learned features.

STEP 7: Evaluate the Original CNN

After training, the model is evaluated on the test set as in Figure 9 and a classification report is generated like in Figure 10.

```
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print("Classification Report: \n", classification_report(y_test,y_pred_classes))
```

Figure 9: Evaluation of Original CNN in Classification Report

Classification Report:

	precision	recall	f1-score	support
0	0.73	0.32	0.44	1000
1	0.75	0.41	0.53	1000
2	0.49	0.14	0.22	1000
3	0.36	0.29	0.32	1000
4	0.36	0.46	0.40	1000
5	0.50	0.22	0.30	1000
6	0.55	0.50	0.52	1000
7	0.29	0.84	0.43	1000
8	0.56	0.68	0.61	1000
9	0.53	0.60	0.56	1000
accuracy			0.45	10000
macro avg	0.51	0.45	0.43	10000
weighted avg	0.51	0.45	0.43	10000

Figure 10: Classification Report of Original CNN Model

The original CNN usually achieves 70% test accuracy after 10 epochs, but in this model the accuracy is only 45% which is not good. The classification report also includes per class precision, recall, and f1-score, providing the detailed view beyond the overall accuracy.

STEP 8: Visualize Prediction

Individual test images as in Figure 11 are plotted alongside their predicted labels to visually assess model performance.

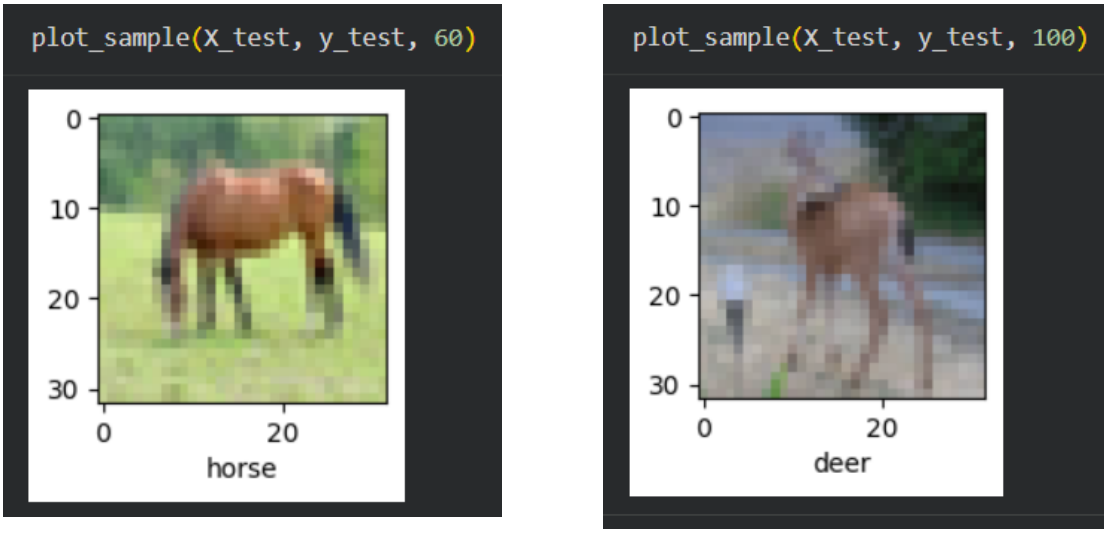

Figure 11: Visualize Prediction

What is the weakest in the current CNN model 10.46 - 11

While the original CNN model improves significantly over the ANN baseline, a critical analysis of its architecture and training process reveals several notable limitations. Some of the weaknesses in the current CNN model were being identified.

- Possible Underfitting from Limited Epochs

  The CNN model training for only 10 epochs may not allow the model to fully converge, especially in the later layers where fine-grained feature patterns are learned. The training loss may still be decreasing at epochs 10, suggesting the model has not yet reached its optimal state.

- Insufficient Depth

  The original CNN has only two convolutional blocks. CIFAR-10 images, while small (32x32), contain complex inter-class variations for example, distinguishing a cat from a deer requires detecting subtle shape and texture patterns. A deeper architecture with more convolutional blocks can learn increasingly abstract representations, improving the discriminative power. The two blocks are insufficient to fully capture these hierarchical features.

- Limited Dense Layer Capacity

  The fully-connected layer before the output has only 64 neurons. After three rounds of convolutional and pooling, the flattened feature vector may contain a few hundred values and compressing them through only 64 neurons limits the model's ability to learn complex decision boundaries.

How to enhance the CNN model?

To overcome the weakness identified, the architecture is upgraded into a modernized pipeline (new_CNN) as shown in Figure xx by introducing and apply the enhancements as follows:

1. Data Augmentation

   This mechanism is applied before the training in order to artificially increase the diversity of the training set as shown in Figure 12. This exposes the model to a wider variety of image presentations without collecting new data as the image is rotated up to 15 degrees, horizontal/vertical shift of 10% and random horizontal flipping during

training on the image.

```
# --- Data Augmentation ---
datagen = ImageDataGenerator(
    rotation_range=15,
    width_shift_range=0.1,
    height_shift_range=0.1,
    horizontal_flip=True
)
datagen.fit(X_train)
```

Figure 12: Data Augmentation mechanisms applied before training

2. Deeper Augmentation with 3 Conv Blocks

   Increasing the number of the convolutional block into 3 blocks with 128 filters is applied as shown in Figure 13. It combined the 'same' padding where the spatial resolution is preserved within each block. This allows deeper layers to operate on richer feature maps.

```
# Block 1
layers.Conv2D(32, (3,3), padding='same', activation='relu', input_shape=(32,32,3)),
BatchNormalization(),
layers.Conv2D(32, (3,3), padding='same', activation='relu'),
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),

# Block 2
layers.Conv2D(64, (3,3), padding='same', activation='relu'),
BatchNormalization(),
layers.Conv2D(64, (3,3), padding='same', activation='relu'),
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),

# Block 3
layers.Conv2D(128, (3,3), padding='same', activation='relu'),
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),
```

Figure 13: 3Conv Block

3. Batch Normalization

   Batch Normalization layers are inserted after each convolutional layer. This layer where it accelerates training convergence, allows use of higher learning rates, acts as a mild regulazier and reduces sensitivity to weight initialization. In the enhanced CNN model, BatchNormalization is applied like Figure 14 after every Conv2D layer and

after the Dense(256) layer.

```
# Block 1
layers.Conv2D(32, (3,3), padding='same'
BatchNormalization(),
layers.Conv2D(32, (3,3), padding='same'
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),

# Block 2
layers.Conv2D(64, (3,3), padding='same'
BatchNormalization(),
layers.Conv2D(64, (3,3), padding='same'
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),

# Block 3
layers.Conv2D(128, (3,3), padding='same
BatchNormalization(),
layers.MaxPooling2D((2,2)),
Dropout(0.25),

layers.Flatten(),
layers.Dense(256, activation='relu'),
BatchNormalization(),
Dropout(0.5),
layers.Dense(10, activation='softmax')
```

Figure 14: BatchNormalization every after Conv2D

4. Extended Training with Augmented Data

   The enhanced model is trained for 20 epochs using the augmented data generator as Figure 15. Training with augmented batches effectively multiplies the number of unique training examples seen per epochs, enabling the model to learn more robust representations.

```
new_cnn.compile(optimizer='adam',
                loss='sparse_categorical_crossentropy',
                metrics=['accuracy'])

# Train with augmented data
new_cnn.fit(datagen.flow(X_train, y_train, batch_size=64),
            epochs=20,
            validation_data=(X_test, y_test))
```

Figure 15: Enhanced Model Training with 20 Epochs

What is the evaluation based on CNN and the new_CNN model?

The CNN model suffers from significant architectural and training limitations that cap its testing accuracy on the CIFAR-10 dataset at 45%. The following code in Figure 16 evaluates and compares both models using multiple metrics including confusion matrix.

```
# Evaluate both
print("=== Original CNN ===")
cnn.evaluate(X_test, y_test)

print("=== Enhanced CNN ===")
new_cnn.evaluate(X_test, y_test)

# Classification report for new_cnn
y_pred_new = new_cnn.predict(X_test)
y_pred_new_classes = [np.argmax(e) for e in y_pred_new]
print(classification_report(y_test, y_pred_new_classes, target_names=classes))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_new_classes)
plt.figure(figsize=(10,8))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=classes, yticklabels=classes)
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Enhanced CNN - Confusion Matrix')
plt.show()
```

Figure 16: Evaluation CNN model and new_CNN model

The comparison in evaluating both models can be seen in Table 1. While the results for the evaluation are visualized as shown in Figure 17 that is based on the output of the source code in Figure 16.

Table 1: Comparison of original CNN and enhanced CNN model

Feature	CNN Model	Enhanced CNN Model
Depth of Architecture	2 Conv blocks	3 Conv blocks
Filter Progression	32 → 64	32 → 64 → 128
Batch Normalization	Not used	Applied after every Conv layer
Dropout	Not used	0.25 after pooling, 0.5 before

Regularization		output
Data Augmentation	Not applied	Rotation, flip and shift through ImageDataGenerator
Dense Layer Size (units)	64	256
Training Epochs	10	20
Test Accuracy	45%	82%

	precision	recall	f1-score	support
airplane	0.86	0.81	0.83	1000
automobile	0.87	0.96	0.91	1000
bird	0.81	0.66	0.73	1000
cat	0.72	0.63	0.67	1000
deer	0.81	0.77	0.79	1000
dog	0.74	0.77	0.76	1000
frog	0.71	0.95	0.81	1000
horse	0.93	0.80	0.86	1000
ship	0.91	0.88	0.89	1000
truck	0.83	0.92	0.87	1000
accuracy			0.82	10000
macro avg	0.82	0.82	0.81	10000
weighted avg	0.82	0.82	0.81	10000

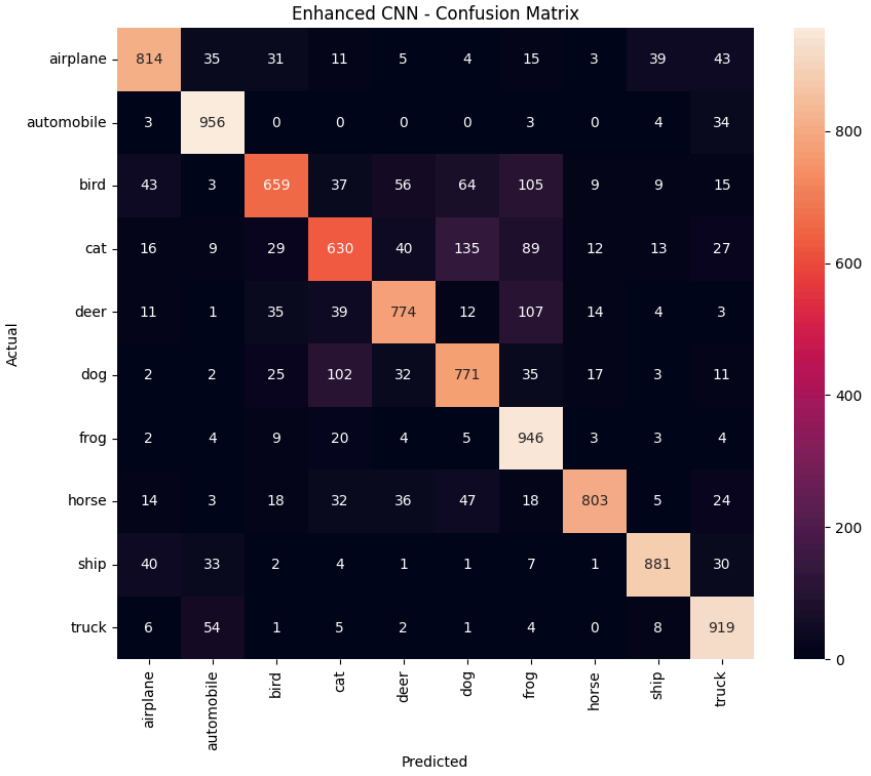

Figure 17: Visualisation of new_CNN Model evaluation

Results and Discussion

The results from the evaluation demonstrate that there is a big contrast in the reliability of the model. The CNN model showed an overfitting as without the implementation of Dropout, the network easily overfits by memorizing the exact pixel placements of the low-resolution images, Besides, the CNN model does not apply BatchNormalization layers which causes drastic shifts in the distribution of inputs to deeper layers, making the learning process mathematically unstable and slow to converge. From the absence of these mechanisms the model trains exclusively on the 50,000 static original images, leaving it highly brittle and unable to accurately classify objects if a test image is even slightly rotated, zoomed, or shifted compared to the training data.

While the newly implemented new_CNN model effectively closed the gap through the enhancement of the model where the implementation of Dropout and Data Augmentation slowed the rise of the accuracy of the training due to its slow learning rate. However, it pushed the validation accuracy over 80% as it accurately classifies objects. Hence, the enhancements applied successfully raised the model's accuracy as the evaluation accuracy improved from approximately to over 80% .

Reflection

1. Nur Firzana Binti Badrus Hisham

   This tutorial offered a step-by-step and hands-on introduction to image classification using CNN model starting from data preprocessing to model evaluation and improvement. The comparison of an ANN baseline with the original CNN made clear the value of convolutional layers, and analyzed the limitations of the model and addressed their weakness through Batch Normalization and generalization in deep learning. The practical experience confirmed that creating a successful CNN model is a process of iteration and requires technical knowledge and critical analysis of outcomes.

2. Nuraisyah Binti Mohd Zikre

   Throughout the completion and the development of this tutorial, it highlights a crucial lesson about deep learning in stacking multiple layers together is not necessary in solving a computer vision problem. However, the problem can be solved by making the training environment become harder (apply Dropout and Data Augmentation

mechanisms to the model) by forcing it to become more intelligent and adaptable. Which also showed a difference between building a model that works in a controlled environment than the environment that generalizes well to the real world data.

3. Wu Chen Xi

This tutorial provided valuable insight into the development and optimization of convolutional neural networks for image classification tasks. Through the process of training, evaluating, and refining different models, I learned that model performance depends not only on architecture design but also on effective training strategies. Techniques such as Batch Normalization, Dropout, and Data Augmentation demonstrated how model robustness can be improved by reducing overfitting and enhancing the model's ability to learn meaningful features from images. The tutorial also emphasized the importance of interpreting evaluation metrics rather than focusing solely on accuracy, as they provide a clearer understanding of model strengths and weaknesses. Overall, this experience strengthened my understanding of deep learning workflows and highlighted that building an effective computer vision model requires continuous experimentation, parameter tuning, and careful analysis of results.